LIBRARY MANAGEMENT SYSTEM

- RUTUJA PRAKASH BHOSALE

BATCH- T330 / DS

Library Management System

A **Library Management System** is a project that simplifies and automates the tasks of managing a library's resources and services. This system ensures the efficient handling of library operations such as cataloging books, managing memberships, and tracking Issued Status

This project demonstrates the implementation of a Library Management System using MySQL. It includes creating and managing tables, performing operations and executing advanced SQL queries. The goal is to showcase skills in database design, manipulation, and querying.

PROJECT AIM

- **Book Management:** Add, update, and remove books from the library's collection. Track book details such as title, category, rental price, availability status, author, and publisher.
- **Membership Management:** Maintain a record of library members, including their names, addresses, registration dates, and issuance history.
- **Employee Management:** Manage library staff, including employee names, positions, salaries, and branch assignments.
- **Book Issuance and Returns**: Track the issuance and return of books by Members. Monitor the status of issued books and ensure timely returns.
- **Branch Management:** Maintain information about library branches, including branch numbers, manager assignments, addresses, and contact details.

OBJECTIVES

1. **Set up the Library Management System Database**:
   Create and populate the database with tables for branches, employees, members, books, issued status, and return status.
2. **CRUD Operation**:
   Perform Create, Read, Update, and Delete operations on the data.
3. **Advanced SQL Queries**:

   Develop complex queries to analyze and retrieve specific data.

ER Diagram For Library Management System

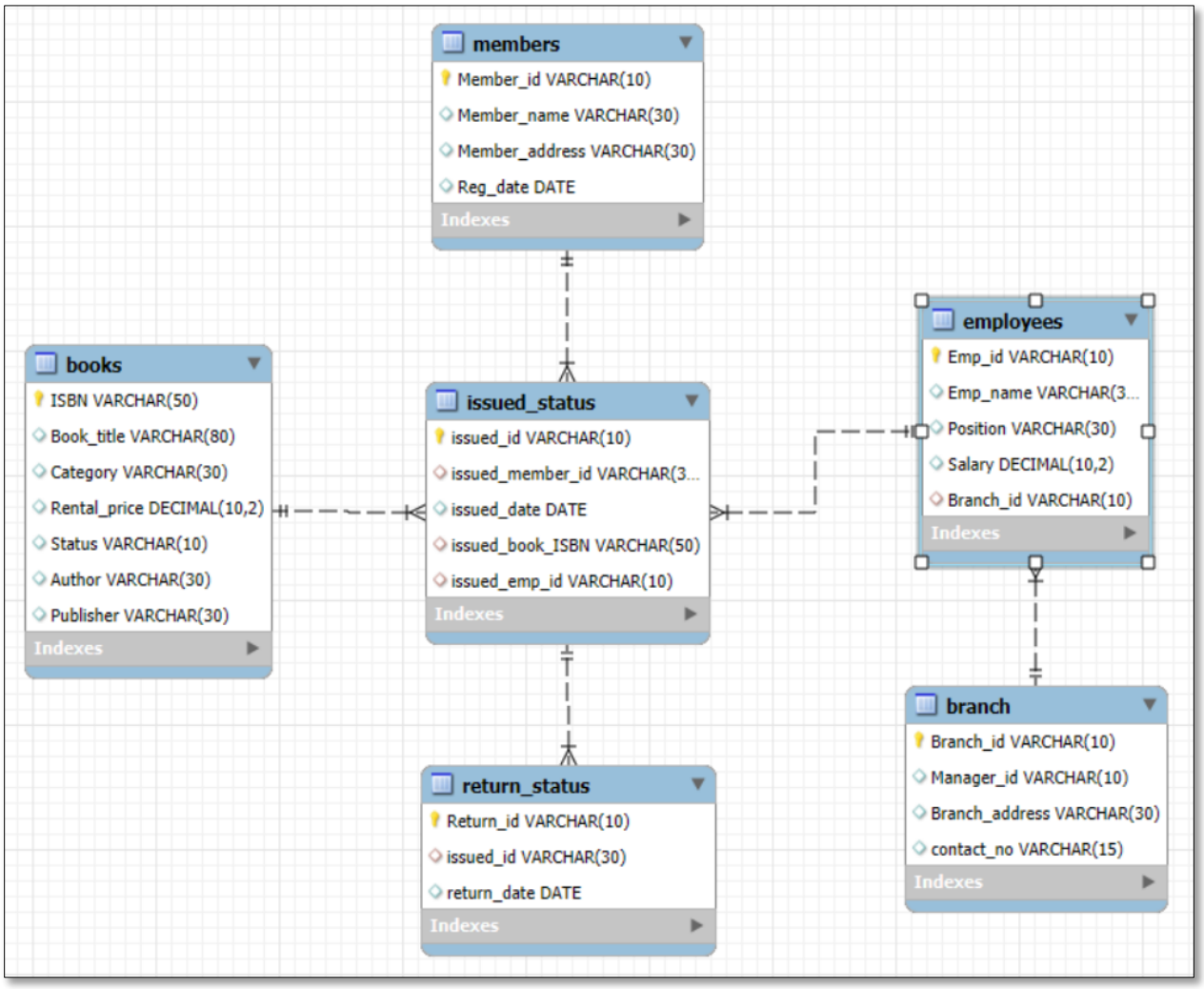


Table Description:

1) **Branch**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
Branch_id	varchar(10)	NO	PRI	NULL	
Manager_id	varchar(10)	YES		NULL	
Branch_address	varchar(30)	YES		NULL	
contact_no	varchar(15)	YES		NULL	

2) **Employees**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
Emp_id	varchar(10)	NO	PRI	NULL	
Emp_name	varchar(30)	YES		NULL	
Position	varchar(30)	YES		NULL	
Salary	decimal(10,2)	YES		NULL	
Branch_id	varchar(10)	YES	MUL	NULL	

3) **Members**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
Member_id	varchar(10)	NO	PRI	NULL	
Member_name	varchar(30)	YES		NULL	
Member_address	varchar(30)	YES		NULL	
Reg_date	date	YES		NULL	

4) **Books**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
ISBN	varchar(50)	NO	PRI	NULL	
Book_title	varchar(80)	YES		NULL	
Category	varchar(30)	YES		NULL	
Rental_price	decimal(10,2)	YES		NULL	
Status	varchar(10)	YES		NULL	
Author	varchar(30)	YES		NULL	
Publisher	varchar(30)	YES		NULL	

5) Issued Status

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
issued_id	varchar(10)	NO	PRI	NULL	
issued_member_id	varchar(30)	YES	MUL	NULL	
issued_date	date	YES		NULL	
issued_book_ISBN	varchar(50)	YES	MUL	NULL	
issued_emp_id	varchar(10)	YES	MUL	NULL	

6) Return Status

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
Return_id	varchar(10)	NO	PRI	NULL	
issued_id	varchar(30)	YES	MUL	NULL	
return_date	date	YES		NULL	

CREATING DATABASE:

CREATE DATABASE library_management_system;
use library_management_system;

Table Creation & Insertion Commands:

1) **Create Table Branch**

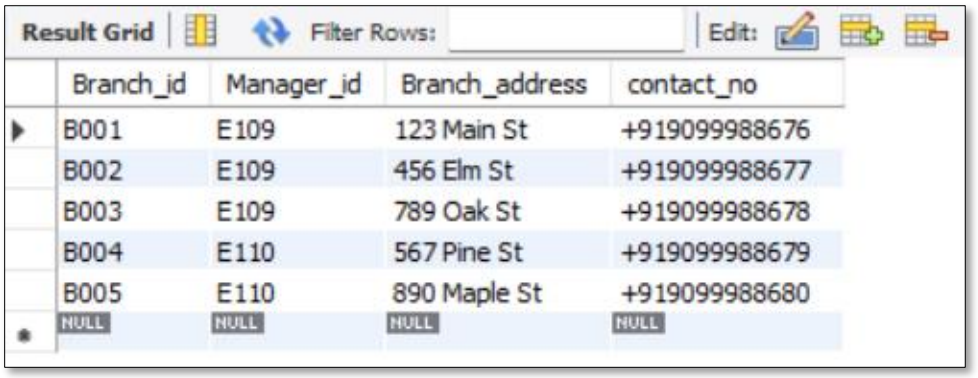
CREATE TABLE Branch
(Branch_id VARCHAR(10) PRIMARY KEY,
Manager_id VARCHAR(10),
Branch_address VARCHAR(30),
Contact_no VARCHAR(15));

Inserting Values into Branch:

INSERT INTO Branch
VALUES
('B001', 'E109', '123 Main St', '+919099988676'),
('B002', 'E109', '456 Elm St', '+919099988677'),
('B003', 'E109', '789 Oak St', '+919099988678'),
('B004', 'E110', '567 Pine St', '+919099988679'),
('B005', 'E110', '890 Maple St', '+919099988680');

SELECT * FROM Branch;

OUTPUT:



Branch_id	Manager_id	Branch_address	contact_no
B001	E109	123 Main St	+919099988676
B002	E109	456 Elm St	+919099988677
B003	E109	789 Oak St	+919099988678
B004	E110	567 Pine St	+919099988679
B005	E110	890 Maple St	+919099988680
NULL	NULL	NULL	NULL

2) **Create Table Employees**

```
CREATE TABLE Employees
(Emp_id VARCHAR(10) PRIMARY KEY,
Emp_name VARCHAR(30),
Position VARCHAR(30),
Salary DECIMAL(10,2),
Branch_id VARCHAR(10),
FOREIGN KEY (Branch_id) REFERENCES  Branch(Branch_id));
```

Inserting Values into Employees:

```
INSERT INTO Employees
VALUES
('E101', 'John Doe', 'Clerk', 60000.00, 'B001'),
('E102', 'Jane Smith', 'Clerk', 45000.00, 'B002'),
('E103', 'Mike Johnson', 'Librarian', 55000.00, 'B001'),
('E104', 'Emily Davis', 'Assistant', 40000.00, 'B001'),
('E105', 'Sarah Brown', 'Assistant', 42000.00, 'B001'),
('E106', 'Michelle Ramirez', 'Assistant', 43000.00, 'B001'),
('E107', 'Michael Thompson', 'Clerk', 62000.00, 'B005'),
('E108', 'Jessica Taylor', 'Clerk', 46000.00, 'B004'),
('E109', 'Daniel Anderson', 'Manager', 57000.00, 'B003'),
('E110', 'Laura Martinez', 'Manager', 41000.00, 'B005'),
('E111', 'Christopher Lee', 'Assistant', 65000.00, 'B005');
```

```
SELECT * FROM Employees;
```

OUTPUT:

Result Grid | Filter Rows: | Edit:

Emp_id	Emp_name	Position	Salary	Branch_id
E101	John Doe	Clerk	60000.00	B001
E102	Jane Smith	Clerk	45000.00	B002
E103	Mike Johnson	Librarian	55000.00	B001
E104	Emily Davis	Assistant	40000.00	B001
E105	Sarah Brown	Assistant	42000.00	B001
E106	Michelle Ramirez	Assistant	43000.00	B001
E107	Michael Thompson	Clerk	62000.00	B005
E108	Jessica Taylor	Clerk	46000.00	B004
E109	Daniel Anderson	Manager	57000.00	B003
E110	Laura Martinez	Manager	41000.00	B005
E111	Christopher Lee	Assistant	65000.00	B005
NULL	NULL	NULL	NULL	NULL

3) **Create Table Members**

CREATE TABLE Members
(Member_id VARCHAR(10) PRIMARY KEY,
Member_name VARCHAR(30),
Member_address VARCHAR(30),
Reg_date DATE);

Inserting Values into Members:

INSERT INTO Members
VALUES
('C101', 'Alice Johnson', '123 Main St', '2021-05-15'),
('C102', 'Bob Smith', '456 Elm St', '2021-06-20'),
('C103', 'Carol Davis', '789 Oak St', '2021-07-10'),
('C104', 'Dave Wilson', '567 Pine St', '2021-08-05'),
('C105', 'Eve Brown', '890 Maple St', '2021-09-25'),
('C106', 'Frank Thomas', '234 Cedar St', '2021-10-15'),
('C107', 'Grace Taylor', '345 Walnut St', '2021-11-20'),
('C108', 'Henry Anderson', '456 Birch St', '2021-12-10'),
('C109', 'Ivy Martinez', '567 Oak St', '2022-01-05'),
('C110', 'Jack Wilson', '678 Pine St', '2022-02-25'),
('C118', 'Sam', '133 Pine St', '2024-06-01'),
('C119', 'John', '143 Main St', '2024-05-01');

SELECT * FROM Members;

OUTPUT:

Result Grid | Filter Rows: | Edit:

Member_id	Member_name	Member_address	Reg_date
C101	Alice Johnson	123 Main St	2021-05-15
C102	Bob Smith	456 Elm St	2021-06-20
C103	Carol Davis	789 Oak St	2021-07-10
C104	Dave Wilson	567 Pine St	2021-08-05
C105	Eve Brown	890 Maple St	2021-09-25
C106	Frank Thomas	234 Cedar St	2021-10-15
C107	Grace Taylor	345 Walnut St	2021-11-20
C108	Henry Anderson	456 Birch St	2021-12-10
C109	Ivy Martinez	567 Oak St	2022-01-05
C110	Jack Wilson	678 Pine St	2022-02-25
C118	Sam	133 Pine St	2024-06-01
C119	John	143 Main St	2024-05-01
NULL	NULL	NULL	NULL

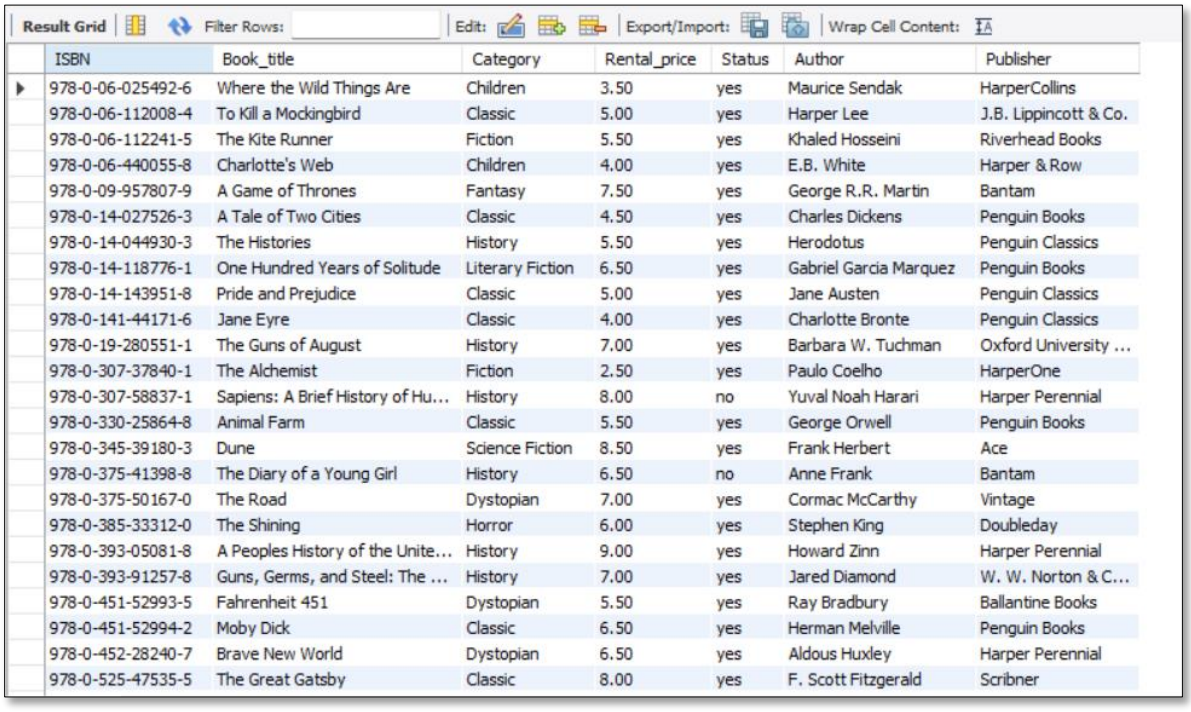
4) Create Table Books

CREATE TABLE Books
(ISBN VARCHAR(50) PRIMARY KEY,
Book_title VARCHAR(80),
Category VARCHAR(30),
Rental_price DECIMAL(10,2),
Status VARCHAR(10),
Author VARCHAR(30),
Publisher VARCHAR(30));

Inserting Values into Books:

INSERT INTO Books
VALUES
('978-0-553-29698-2', 'The Catcher in the Rye', 'Classic', 7.00, 'yes', 'J.D. Salinger', 'Little, Brown and Company'),
('978-0-330-25864-8', 'Animal Farm', 'Classic', 5.50, 'yes', 'George Orwell', 'Penguin Books'),
('978-0-14-118776-1', 'One Hundred Years of Solitude', 'Literary Fiction', 6.50, 'yes', 'Gabriel Garcia Marquez', 'Penguin Books'),
.
.
SELECT * FROM Books;

OUTPUT:



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ISBN	Book_title	Category	Rental_price	Status	Author	Publisher
978-0-06-025492-6	Where the Wild Things Are	Children	3.50	yes	Maurice Sendak	HarperCollins
978-0-06-112008-4	To Kill a Mockingbird	Classic	5.00	yes	Harper Lee	J.B. Lippincott & Co.
978-0-06-112241-5	The Kite Runner	Fiction	5.50	yes	Khaled Hosseini	Riverhead Books
978-0-06-440055-8	Charlotte's Web	Children	4.00	yes	E.B. White	Harper & Row
978-0-09-957807-9	A Game of Thrones	Fantasy	7.50	yes	George R.R. Martin	Bantam
978-0-14-027526-3	A Tale of Two Cities	Classic	4.50	yes	Charles Dickens	Penguin Books
978-0-14-044930-3	The Histories	History	5.50	yes	Herodotus	Penguin Classics
978-0-14-118776-1	One Hundred Years of Solitude	Literary Fiction	6.50	yes	Gabriel Garcia Marquez	Penguin Books
978-0-14-143951-8	Pride and Prejudice	Classic	5.00	yes	Jane Austen	Penguin Classics
978-0-141-44171-6	Jane Eyre	Classic	4.00	yes	Charlotte Bronte	Penguin Classics
978-0-19-280551-1	The Guns of August	History	7.00	yes	Barbara W. Tuchman	Oxford University ...
978-0-307-37840-1	The Alchemist	Fiction	2.50	yes	Paulo Coelho	HarperOne
978-0-307-58837-1	Sapiens: A Brief History of Hu...	History	8.00	no	Yuval Noah Harari	Harper Perennial
978-0-330-25864-8	Animal Farm	Classic	5.50	yes	George Orwell	Penguin Books
978-0-345-39180-3	Dune	Science Fiction	8.50	yes	Frank Herbert	Ace
978-0-375-41398-8	The Diary of a Young Girl	History	6.50	no	Anne Frank	Bantam
978-0-375-50167-0	The Road	Dystopian	7.00	yes	Cormac McCarthy	Vintage
978-0-385-33312-0	The Shining	Horror	6.00	yes	Stephen King	Doubleday
978-0-393-05081-8	A Peoples History of the Unite...	History	9.00	yes	Howard Zinn	Harper Perennial
978-0-393-91257-8	Guns, Germs, and Steel: The ...	History	7.00	yes	Jared Diamond	W. W. Norton & C...
978-0-451-52993-5	Fahrenheit 451	Dystopian	5.50	yes	Ray Bradbury	Ballantine Books
978-0-451-52994-2	Moby Dick	Classic	6.50	yes	Herman Melville	Penguin Books
978-0-452-28240-7	Brave New World	Dystopian	6.50	yes	Aldous Huxley	Harper Perennial
978-0-525-47535-5	The Great Gatsby	Classic	8.00	yes	F. Scott Fitzgerald	Scribner

5) **Create Table issued_status**

```
CREATE TABLE issued_status
(issued_id VARCHAR(10) PRIMARY KEY,
issued_member_id VARCHAR(30),
issued_date DATE,
issued_book_ISBN VARCHAR(50),
issued_emp_id VARCHAR(10),
FOREIGN KEY (issued_member_id) REFERENCES Members(Member_id),
FOREIGN KEY (issued_emp_id) REFERENCES Employees(Emp_id),
FOREIGN KEY (issued_book_isbn) REFERENCES Books(ISBN) );
```

Inserting Values into issued_status:

```
INSERT INTO issued_status
VALUES
('IS106', 'C106', '2024-03-10', '978-0-330-25864-8', 'E104'),
('IS107', 'C107', '2024-03-11', '978-0-14-118776-1', 'E104'),
('IS108', 'C108', '2024-03-12', '978-0-525-47535-5', 'E104'),
('IS109', 'C109', '2024-03-13', '978-0-141-44171-6', 'E105'),
('IS110', 'C110', '2024-03-14', '978-0-307-37840-1', 'E105'),
.
.
SELECT * FROM issued_status;
```

OUTPUT:

Result Grid | Filter Rows: | Edit: | Export/Import:

issued_id	issued_member_id	issued_date	issued_book_ISBN	issued_emp_id
IS106	C106	2024-03-10	978-0-330-25864-8	E104
IS107	C107	2024-03-11	978-0-14-118776-1	E104
IS108	C108	2024-03-12	978-0-525-47535-5	E104
IS109	C109	2024-03-13	978-0-141-44171-6	E105
IS110	C110	2024-03-14	978-0-307-37840-1	E105
IS111	C109	2024-03-15	978-0-679-76489-8	E105
IS112	C109	2024-03-16	978-0-09-957807-9	E106
IS113	C109	2024-03-17	978-0-393-05081-8	E106
IS114	C109	2024-03-18	978-0-19-280551-1	E106
IS115	C109	2024-03-19	978-0-14-044930-3	E107
IS116	C110	2024-03-20	978-0-393-91257-8	E107
IS117	C110	2024-03-21	978-0-679-64115-3	E107
IS118	C101	2024-03-22	978-0-14-143951-8	E108
IS119	C110	2024-03-23	978-0-452-28240-7	E108
IS120	C110	2024-03-24	978-0-670-81302-4	E108
IS121	C102	2024-03-25	978-0-385-33312-0	E109
IS122	C102	2024-03-26	978-0-451-52993-5	E109
IS123	C103	2024-03-27	978-0-345-39180-3	E109
IS124	C104	2024-03-28	978-0-06-025492-6	E110
IS125	C105	2024-03-29	978-0-06-112241-5	E110
IS126	C105	2024-03-30	978-0-06-440055-8	E110
IS127	C105	2024-03-31	978-0-679-77644-3	E110

6) **Create Table Return_status**

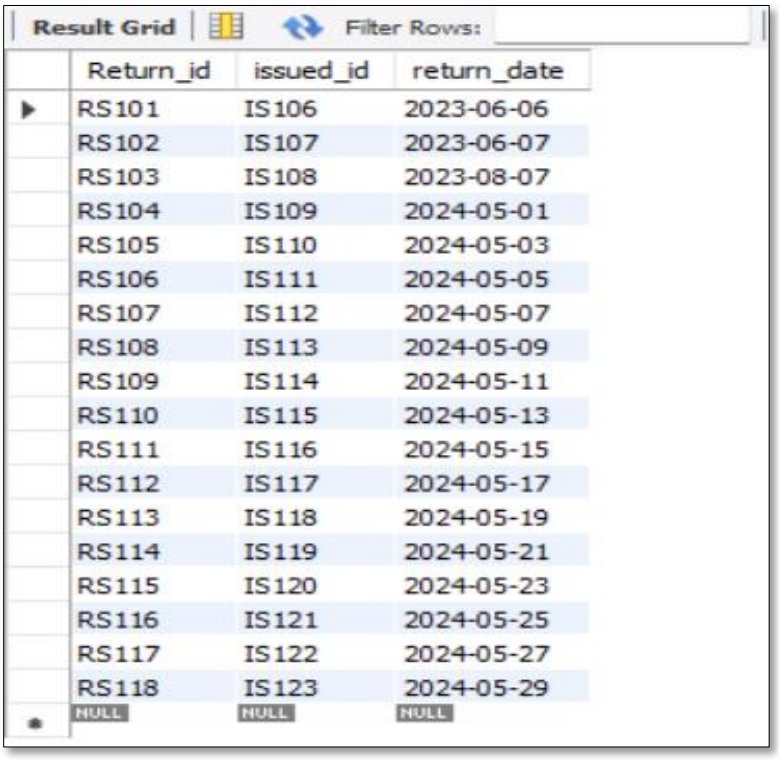
```
CREATE TABLE Return_status
(Return_id VARCHAR(10) PRIMARY KEY,
issued_id VARCHAR(30),
return_date DATE,
FOREIGN KEY (issued_id) REFERENCES issued_status(issued_id));
```

Inserting Values Return_status:

```
INSERT INTO Return_status
VALUES
('RS101', 'IS106', '2023-06-06'),
('RS102', 'IS107', '2023-06-07'),
('RS103', 'IS108', '2023-08-07'),
.
.
```

```
SELECT * FROM Return_status;
```

OUTPUT:



Result Grid | Filter Rows:

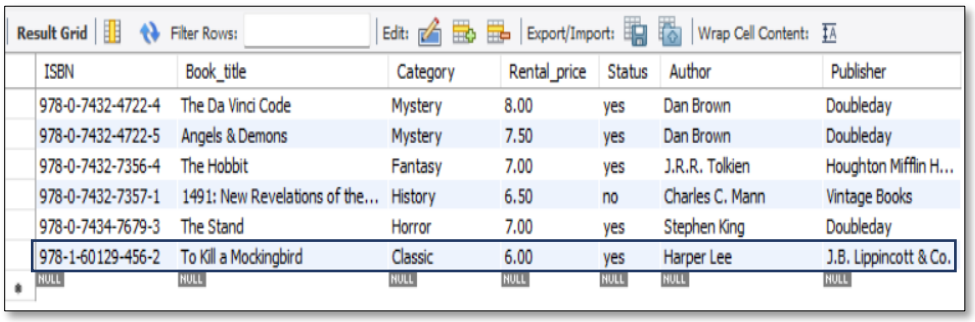
Return_id	issued_id	return_date
RS101	IS106	2023-06-06
RS102	IS107	2023-06-07
RS103	IS108	2023-08-07
RS104	IS109	2024-05-01
RS105	IS110	2024-05-03
RS106	IS111	2024-05-05
RS107	IS112	2024-05-07
RS108	IS113	2024-05-09
RS109	IS114	2024-05-11
RS110	IS115	2024-05-13
RS111	IS116	2024-05-15
RS112	IS117	2024-05-17
RS113	IS118	2024-05-19
RS114	IS119	2024-05-21
RS115	IS120	2024-05-23
RS116	IS121	2024-05-25
RS117	IS122	2024-05-27
RS118	IS123	2024-05-29
NULL	NULL	NULL

BASIC QUESTIONS

1. **Create a New Book Record – (‘978-1-60129-456-2', 'To Kill a Mockingbird', 'Classic', 6.00, 'yes', 'Harper Lee', 'J.B. Lippincott & Co.’)**

   INSERT INTO Books
   VALUES('978-1-60129-456-2', 'To Kill a Mockingbird', 'Classic', 6.00, 'yes', 'Harper Lee', 'J.B. Lippincott & Co.');
   SELECT * FROM Books;

   OUTPUT:

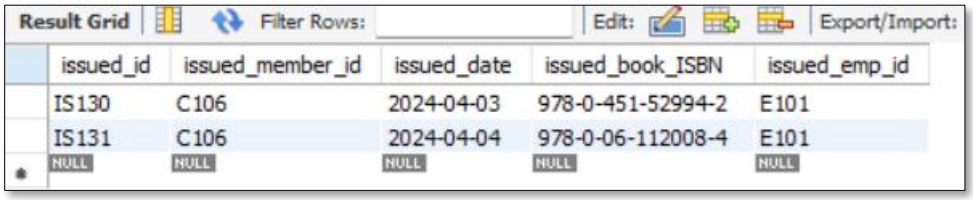


ISBN	Book_title	Category	Rental_price	Status	Author	Publisher
978-0-7432-4722-4	The Da Vinci Code	Mystery	8.00	yes	Dan Brown	Doubleday
978-0-7432-4722-5	Angels & Demons	Mystery	7.50	yes	Dan Brown	Doubleday
978-0-7432-7356-4	The Hobbit	Fantasy	7.00	yes	J.R.R. Tolkien	Houghton Mifflin H...
978-0-7432-7357-1	1491: New Revelations of the...	History	6.50	no	Charles C. Mann	Vintage Books
978-0-7434-7679-3	The Stand	Horror	7.00	yes	Stephen King	Doubleday
978-1-60129-456-2	To Kill a Mockingbird	Classic	6.00	yes	Harper Lee	J.B. Lippincott & Co.

2. **Retrieve All Issued status Issued by a Specific Employee**

   SELECT * **FROM** issued_status **WHERE** issued_emp_id = 'E101';

   OUTPUT:

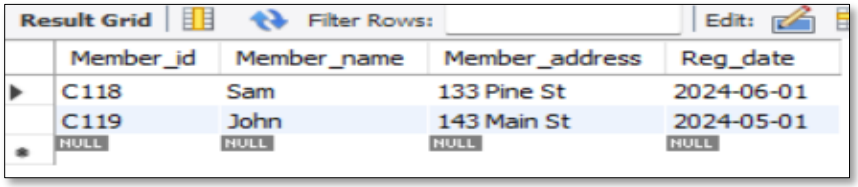


issued_id	issued_member_id	issued_date	issued_book_ISBN	issued_emp_id
IS130	C106	2024-04-03	978-0-451-52994-2	E101
IS131	C106	2024-04-04	978-0-06-112008-4	E101

3. **List Members Who Registered in the Last 365 Days**

   SELECT * **FROM** Members **WHERE** reg_date >= **date_sub**(curdate(),INTERVAL 365 day) ;

   OUTPUT:

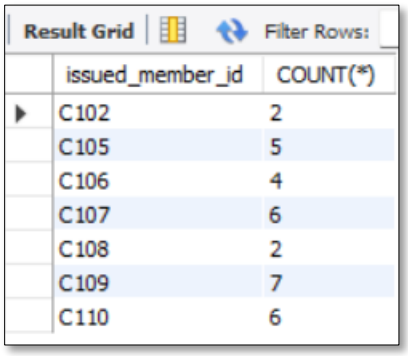


Member_id	Member_name	Member_address	Reg_date
C118	Sam	133 Pine St	2024-06-01
C119	John	143 Main St	2024-05-01

4. List Members Who Have Issued More Than One Book

```
SELECT issued_member_id , COUNT(*) FROM issued_status GROUP BY
issued_member_id HAVING COUNT(*) > 1;
```

OUTPUT:



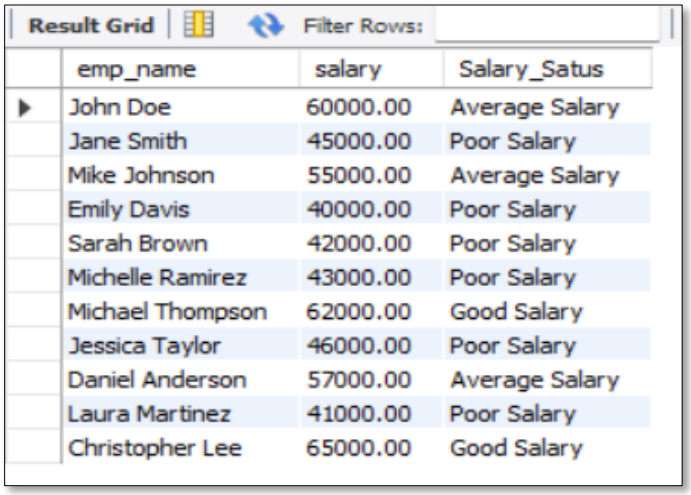
Result Grid | Filter Rows:

issued_member_id	COUNT(*)
C102	2
C105	5
C106	4
C107	6
C108	2
C109	7
C110	6

5. List Employees who have Salary wise Salary Status

```
SELECT emp_name,salary,
CASE
when salary > 60000 then 'Good Salary'
when salary > 50000 then 'Average Salary'
else 'Poor Salary'
END as Salary_Satus
FROM Employees;
```

OUTPUT:



Result Grid | Filter Rows:

emp_name	salary	Salary_Satus
John Doe	60000.00	Average Salary
Jane Smith	45000.00	Poor Salary
Mike Johnson	55000.00	Average Salary
Emily Davis	40000.00	Poor Salary
Sarah Brown	42000.00	Poor Salary
Michelle Ramirez	43000.00	Poor Salary
Michael Thompson	62000.00	Good Salary
Jessica Taylor	46000.00	Poor Salary
Daniel Anderson	57000.00	Average Salary
Laura Martinez	41000.00	Poor Salary
Christopher Lee	65000.00	Good Salary

6. **Display the Books who's Book Titles are starting with "The"**

SELECT Book_title **FROM** Books **WHERE** Book_title **LIKE** 'The%';

OUTPUT:

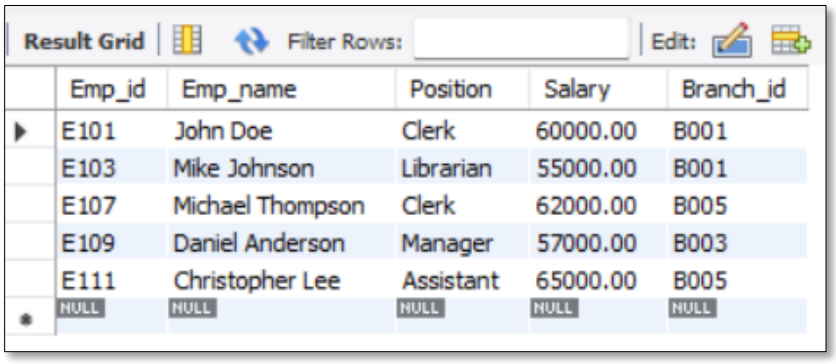
Result Grid | Filter Ro

Book_title
The Kite Runner
The Histories
The Guns of August
The Alchemist
The Diary of a Young Girl
The Road
The Shining
The Great Gatsby
The Catcher in the Rye
The Road
The Da Vinci Code
The Hobbit
The Stand

7. **Display the Employees Salary Between 50000 to 70000**

SELECT * **FROM** Employees **WHERE** Salary **BETWEEN** 50000 **AND** 70000;

OUTPUT:



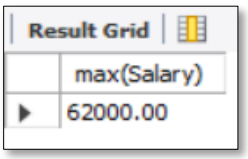
Result Grid | Filter Rows: | Edit:

Emp_id	Emp_name	Position	Salary	Branch_id
E101	John Doe	Clerk	60000.00	B001
E103	Mike Johnson	Librarian	55000.00	B001
E107	Michael Thompson	Clerk	62000.00	B005
E109	Daniel Anderson	Manager	57000.00	B003
E111	Christopher Lee	Assistant	65000.00	B005
NULL	NULL	NULL	NULL	NULL

SUB-QUERIES

1. **2nd Highest Salary of the Employee**

   SELECT max(Salary) **FROM** Employees **WHERE** Salary < (SELECT max(Salary) FROM Employees);

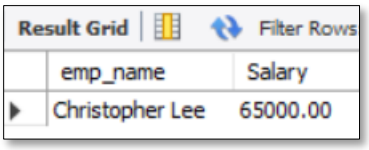


max(Salary)
62000.00

2. **Write a query to find the employee who are earning the highest salary**

   SELECT emp_name,Salary **FROM** Employees **WHERE** Salary = (SELECT max(Salary) FROM Employees);

   OUTPUT:

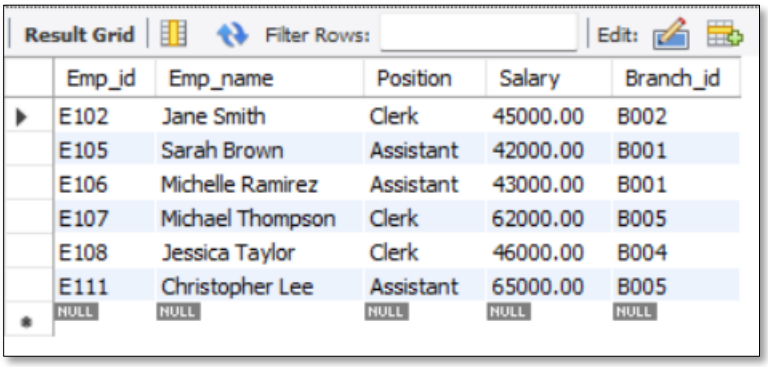


emp_name	Salary
Christopher Lee	65000.00

3. **Display the Employees Who have same position As "John Doe" and "Emily Davis"**

   SELECT * **FROM** Employees **WHERE** Position **IN** (SELECT Position FROM Employees **WHERE** Emp_name="John Doe" OR Emp_name="Emily Davis") **AND** (Emp_name<>"John Doe" AND Emp_name<> "Emily Davis");

   OUTPUT:

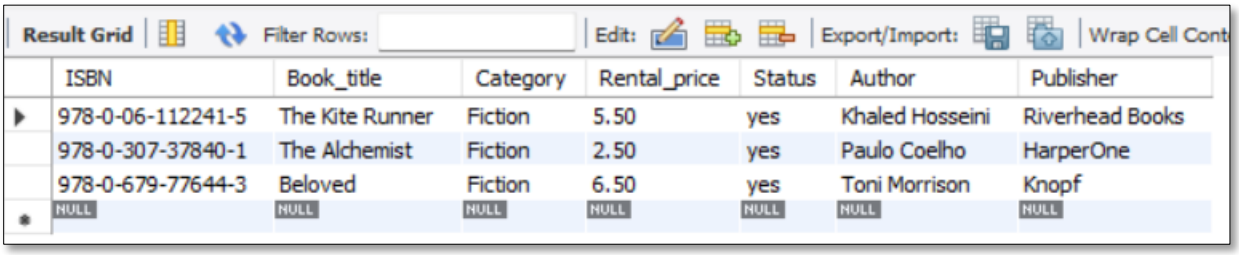


Emp_id	Emp_name	Position	Salary	Branch_id
E102	Jane Smith	Clerk	45000.00	B002
E105	Sarah Brown	Assistant	42000.00	B001
E106	Michelle Ramirez	Assistant	43000.00	B001
E107	Michael Thompson	Clerk	62000.00	B005
E108	Jessica Taylor	Clerk	46000.00	B004
E111	Christopher Lee	Assistant	65000.00	B005
NULL	NULL	NULL	NULL	NULL

4. **Display the Books detailed who have same Category as the Book “The Alchemist”**

   SELECT * **FROM** Books **WHERE** Category = (SELECT Category FROM Books WHERE Book_title="The Alchemist");

   OUTPUT:



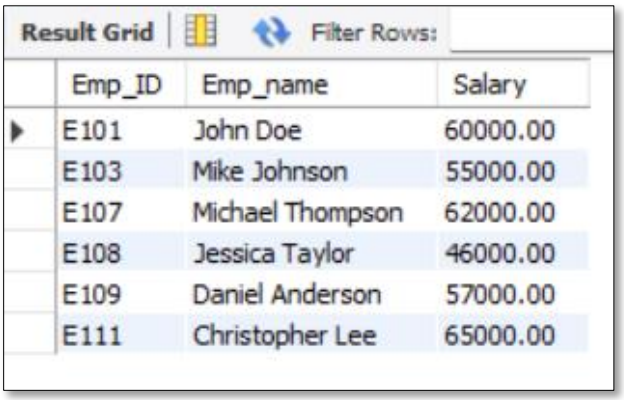
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Cont

ISBN	Book_title	Category	Rental_price	Status	Author	Publisher
978-0-06-112241-5	The Kite Runner	Fiction	5.50	yes	Khaled Hosseini	Riverhead Books
978-0-307-37840-1	The Alchemist	Fiction	2.50	yes	Paulo Coelho	HarperOne
978-0-679-77644-3	Beloved	Fiction	6.50	yes	Toni Morrison	Knopf
NULL	NULL	NULL	NULL	NULL	NULL	NULL

5. **Write a query to find the employees whose salary is greater than at least on employee in Emp_ID = 102**

   SELECT Emp_ID,Emp_name,Salary **FROM** Employees **WHERE** Salary > **ANY**(SELECT Salary FROM Employees WHERE Emp_ID = "E102");

   OUTPUT:



Result Grid | Filter Rows:

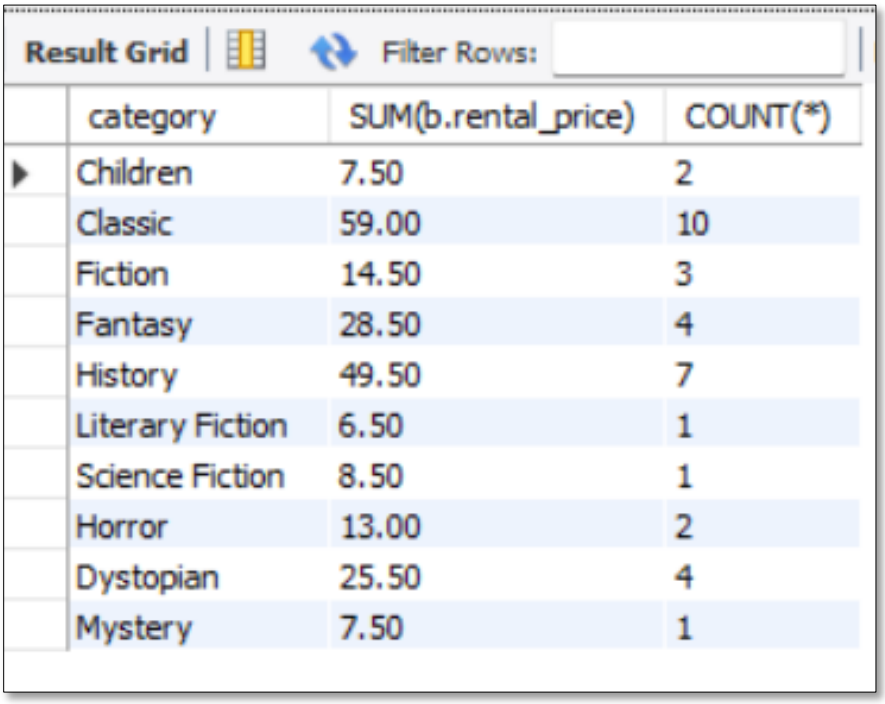
Emp_ID	Emp_name	Salary
E101	John Doe	60000.00
E103	Mike Johnson	55000.00
E107	Michael Thompson	62000.00
E108	Jessica Taylor	46000.00
E109	Daniel Anderson	57000.00
E111	Christopher Lee	65000.00

JOINS

1. Find Total Rental Income by Category

```
SELECT b.category,SUM(b.rental_price),COUNT(*) FROM
issued_status as ist
INNER JOIN
Books as b
ON b.isbn = ist.issued_book_isbn
GROUP BY Category;
```

OUTPUT:



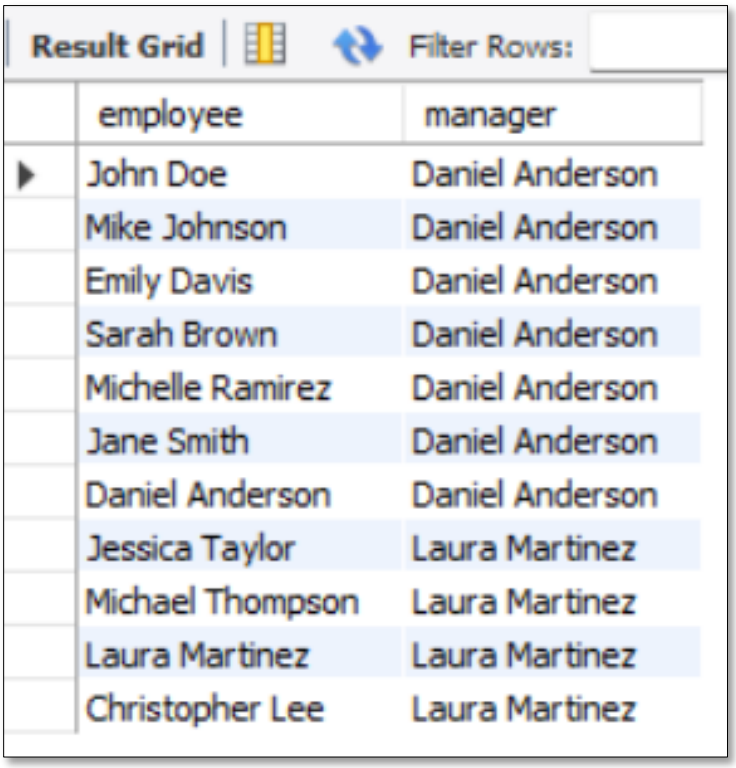
Result Grid | Filter Rows:

category	SUM(b.rental_price)	COUNT(*)
Children	7.50	2
Classic	59.00	10
Fiction	14.50	3
Fantasy	28.50	4
History	49.50	7
Literary Fiction	6.50	1
Science Fiction	8.50	1
Horror	13.00	2
Dystopian	25.50	4
Mystery	7.50	1

2. Display Employees with Their Branch Manager's Name

```
SELECT e1.emp_name as employee,e2.emp_name as manager FROM
employees as e1
JOIN
branch as b
ON e1.branch_id = b.branch_id
JOIN
employees as e2
ON e2.emp_id = b.manager_id;
```

OUTPUT:



Result Grid | Filter Rows:

employee	manager
John Doe	Daniel Anderson
Mike Johnson	Daniel Anderson
Emily Davis	Daniel Anderson
Sarah Brown	Daniel Anderson
Michelle Ramirez	Daniel Anderson
Jane Smith	Daniel Anderson
Daniel Anderson	Daniel Anderson
Jessica Taylor	Laura Martinez
Michael Thompson	Laura Martinez
Laura Martinez	Laura Martinez
Christopher Lee	Laura Martinez

3. Retrieve the List of Books Not Yet Returned

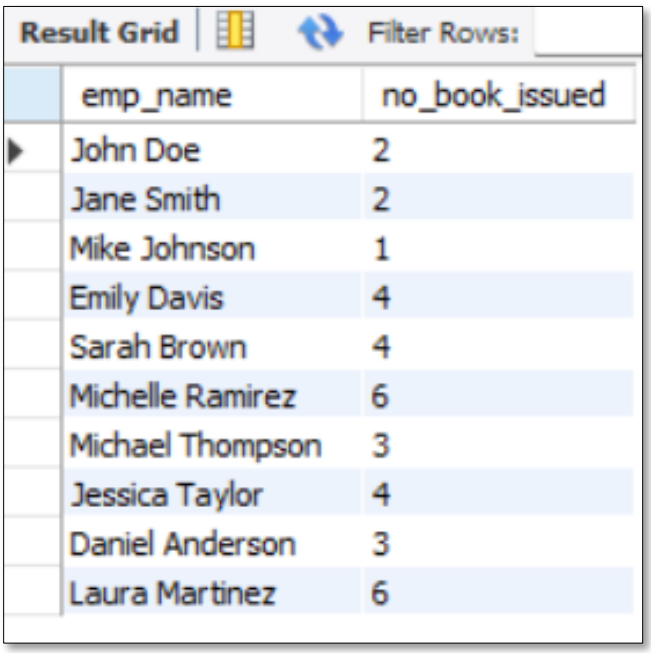
```
SELECT Books.Book_title FROM
issued_status as ist
LEFT JOIN
Books
ON ist.Issued_book_isbn = Books.ISBN
LEFT JOIN
return_status as rs
ON rs.issued_id = ist.issued_id
WHERE rs.return_id IS NULL;
```

OUTPUT:

4. **Find Employees with the Most Book Issues Processed**

```
SELECT e.emp_name,COUNT(ist.issued_id) as no_book_issued FROM
issued_status as ist
JOIN
employees as e
ON e.emp_id = ist.issued_emp_id
GROUP BY e.emp_name;
```

OUTPUT:



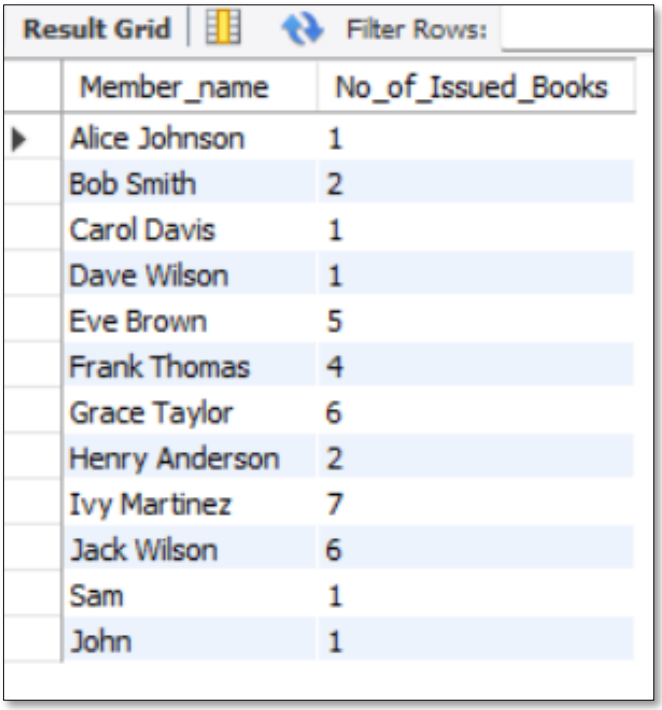
Result Grid | Filter Rows:

emp_name	no_book_issued
John Doe	2
Jane Smith	2
Mike Johnson	1
Emily Davis	4
Sarah Brown	4
Michelle Ramirez	6
Michael Thompson	3
Jessica Taylor	4
Daniel Anderson	3
Laura Martinez	6

5. Display how Many Books Issued by each Members

```
SELECT m.Member_name,count(*) as No_of_Issued_Books FROM
Members as m
LEFT JOIN
issued_status as ist
ON m.member_id = ist.issued_member_id
GROUP BY m.Member_name;
```

OUTPUT:



Result Grid | Filter Rows:

Member_name	No_of_Issued_Books
Alice Johnson	1
Bob Smith	2
Carol Davis	1
Dave Wilson	1
Eve Brown	5
Frank Thomas	4
Grace Taylor	6
Henry Anderson	2
Ivy Martinez	7
Jack Wilson	6
Sam	1
John	1

CONCLUSION

The Library Management System Database project has successfully designed and implemented a comprehensive database system that effectively manages and stores information related to the Library operations. The system has achieved its objectives by providing a user-friendly interface, improving data accuracy and accessibility. With its robust and scalable design, the system is well-positioned to support the Library Management future growth and expansion. This project demonstrates the application of SQL skills in creating and managing a library management system. It includes database setup, data manipulation, and advanced querying, providing a solid foundation for data management and analysis.